Final Project Report

Smart Food Waste Reduction & Redistribution System

Course Name:	Software Engineering Design Capstone Project
Course Code:	SE 331 (F1)
Supervisor:	Rahat Uddin Azad, Lecturer, DIU, SWE
University:	Daffodil International University

Group: Social Warrior

Group Members

ID	Name
0242310005341219	Md.Mahbub Alam
0242310005341015	Tanvir Ahmed
0242310005341171	Md Muhtasim Fuad Rahat

1. Executive Summary

Food waste is a critical global problem, yet millions of people remain food-insecure. The Smart Food Waste Reduction & Redistribution System addresses the disconnect between food surplus and food need by providing a centralized, automated web platform that bridges food providers (restaurants, hotels, households, community centers) with verified food distributors (NGOs, volunteers).

The existing manual processes — phone calls, social media posts — are slow, decentralized, and ineffective. By the time a distributor is located and contacted, food has often already expired. Our system eliminates this latency through three core value pillars: speed (distributors within a 5km radius are notified within 10 seconds of a food post), accountability (every transaction is recorded and distribution proofs are monitored by admins), and trust (all users are verified through legal document submission before being granted access). The result is a transparent, real-time redistribution pipeline that turns wasted food into a community resource.

Technology Stack:

Layer	Technology
Frontend	React.js, Tailwind CSS, JS, HTML, CSS
Backend	Node.js, Express.js
Database	MongoDB
Authentication	JSON Web Tokens (JWT)
Architecture	RESTFul API, Component based UI

Tanvir Ahmed
ID: 0242310005341015
Role: Frontend Developer & UI/UX Designer

Md Muhtasim Fuad Rahat
ID: 0242310005341171
Role: Project Manager & Backend Developer

Md Mahbub Alam
ID: 0242310005341219
Role: System Analyst, Quality Assurance & Tester

2. Introduction

2.1. Problem Statement

Every day, large quantities of edible food are discarded by restaurants, hotels, households, and community events — not because the food is inedible, but because there is no efficient mechanism to connect surplus food with people who need it. The current redistribution process relies entirely on manual coordination: phone calls, WhatsApp messages, or social media posts. This approach is slow, unstructured, and decentralized. The inevitable latency between a food post and a distributor's response frequently results in food expiring before it can be collected, making the entire effort futile. There is no system to track pickups, verify distributors, or hold anyone accountable for whether redistributed food actually reaches those in need.

2.2. Project Objectives

- Build a centralized web platform that connects verified food providers and distributors in real time, eliminating the need for manual coordination.
- Implement a geolocation-based notification system that alerts distributors within a 5km radius within 10 seconds of a food post going live.
- Enforce accountability at every stage of the redistribution process — from claim and pickup confirmation to mandatory upload of distribution proofs.
- Develop a role-based multi-user system with separate, purpose-built dashboards for Providers, Distributors, and Admins.
- Provide Admin-level oversight tools including document verification, activity monitoring, and the ability to block users who violate distribution proof requirements.

2.3. System Boundaries

In- Scope (What does the system do):

- User registration and secure login for Providers, Distributors, and Admin
- Admin verification of legal documents before granting full account access
- Providers can post surplus food with details (type, quantity, expiry time, location)
- Distributors can browse available food listings within their area
- Real-time push notifications to Distributors within a 5km radius of a food post
- Distributors can submit a pickup request; Providers can approve or reject it
- Both parties confirm physical pickup, creating a verifiable transaction record
- Distributors must upload distribution proof (photo or report) after delivery
- Admin dashboard for monitoring proofs, managing users, and generating reports
- Automated blocking of Distributors with two or more missing distribution proofs

Out of Scope (What the system does not do):

- No mobile application; the platform is web-only
- No transportation or delivery tracking service
- No financial transactions of any kind

2.4. Stakeholder Analysis

Stakeholder	Description	Key Needs Within the System
Food Providers	Restaurants, hotels, households, and community centers with surplus food	Ability to post food quickly, review distributor history before approving requests, confirm pickups, and maintain a clean transaction record
Food Distributors	NGOs, volunteers, and community workers who collect and redistribute food	Receive timely nearby food alerts, submit pickup requests, get fast approval/rejection notifications, and upload distribution proofs easily
System Admin	Platform super-user responsible for platform integrity	Tools to verify user documents, monitor distribution proofs, flag fraudulent activity, block non-compliant users, and access platform-wide reports

3. Requirement Specification

3.1. Functional Requirements (FR)

Req. ID	Feature Name	Description	Priority	Status
FR-1	Registration	The system shall allow Providers and Distributors to create an account	High	Implemented
FR-2	Submit Documents	Users shall submit legal documents for admin authentication and verification during registration	High	Implemented
FR-3	Login	The system shall allow Providers, Distributors, and Admin to log in to their respective accounts	High	Implemented
FR-4	Role-Based Dashboard	The system shall provide separate dashboards for Providers, Distributors, and Admin with role-specific features	High	Implemented
FR-5	Post Food	Providers shall post surplus food details including type, quantity, location, and expiry time	High	Implemented
FR-6	Get Request Notification (Provider)	Providers shall receive a notification when a Distributor submits a pickup request	High	Implemented
FR-7	View Distributor History	Providers shall be able to view a Distributor's transaction history and distribution proofs before making a decision	High	Implemented

FR-8	Accept or Reject Request	Providers shall approve or reject pickup requests based on Distributor history and proofs	High	Implemented
FR-9	Confirm Pickup	Both Providers and Distributors shall confirm and update status when food is physically picked up	High	Implemented
FR-10	Logout	The system shall allow all users to securely log out of their accounts	High	Implemented
FR-11	Nearby Food Notification	Distributors shall be alerted when food is posted within a 5km radius	High	Implemented
FR-12	View Food	Distributors shall browse available food listings with search and filter functionality	High	Implemented
FR-13	Request for Food	Distributors shall be able to submit a pickup request for available food posts	High	Implemented
FR-14	Request Approval/Rejection Notification	Distributors shall receive a notification when a Provider approves or rejects their request	High	Implemented
FR-15	Upload Proofs	Distributors shall upload food distribution proof (photo/report) after completing delivery	High	Implemented
FR-16	Verify User Documents	Admin shall review and verify submitted legal documents to authenticate user accounts	High	Implemented
FR-17	Admin Dashboard	Admin shall have a dashboard for user activity monitoring, statistics, and food listing management	High	Implemented
FR-18	Monitor Proofs	Admin shall review and validate Distributor distribution proofs	Medium	Implemented
FR-19	Block Users	Admin will block distributor if there is 2 or more days of delay to upload proofs.	High	Implemented
FR-20	Send Notifications	The system shall send automated notifications for nearby food posts and request status updates	High	Implemented

3.2. Non- Functional Requirements

Performance

The system is designed for fast response under real-world usage. Common user actions such as login, food posting, and request submission complete within 2 seconds, while food listings load within 3 seconds. It has been tested under 1000 concurrent users with stable performance.

Security

The system uses JWT-based authentication and securely stores passwords using hashing techniques. Sensitive data is protected from unauthorized access, and role-based access control (RBAC) ensures users can only access features relevant to their roles.

Usability

The platform provides a clean and user-friendly interface built with React.js and Tailwind CSS. Separate dashboards for each role improve usability and reduce complexity.

Availability

The system is deployed on Vercel and is available 24/7, except for scheduled maintenance, which is limited to 2 hours per month.

Scalability

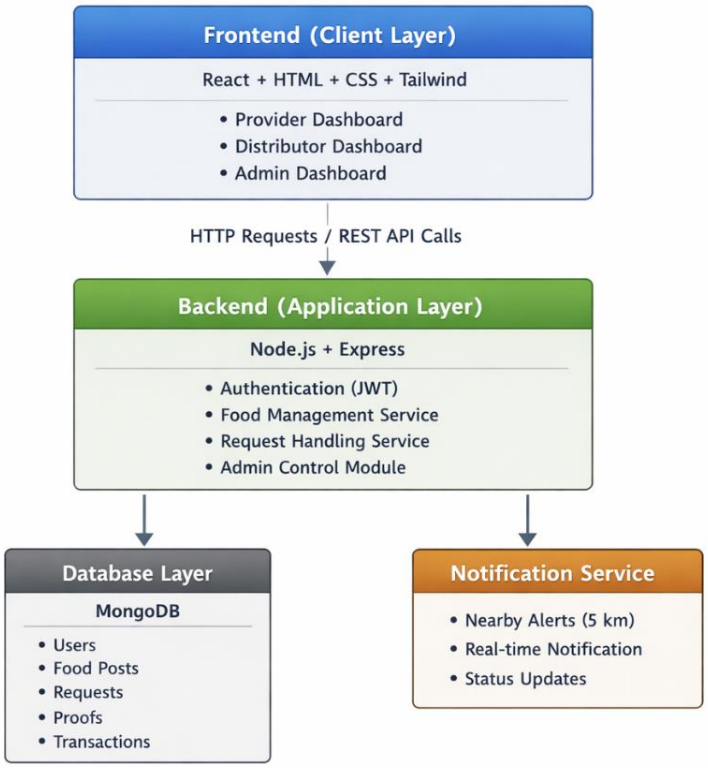
Built with Node.js and MongoDB, the system supports at least 1,000 concurrent users and is designed for future horizontal scaling.

4. System Design & Architecture

4.1. System Architecture Diagram

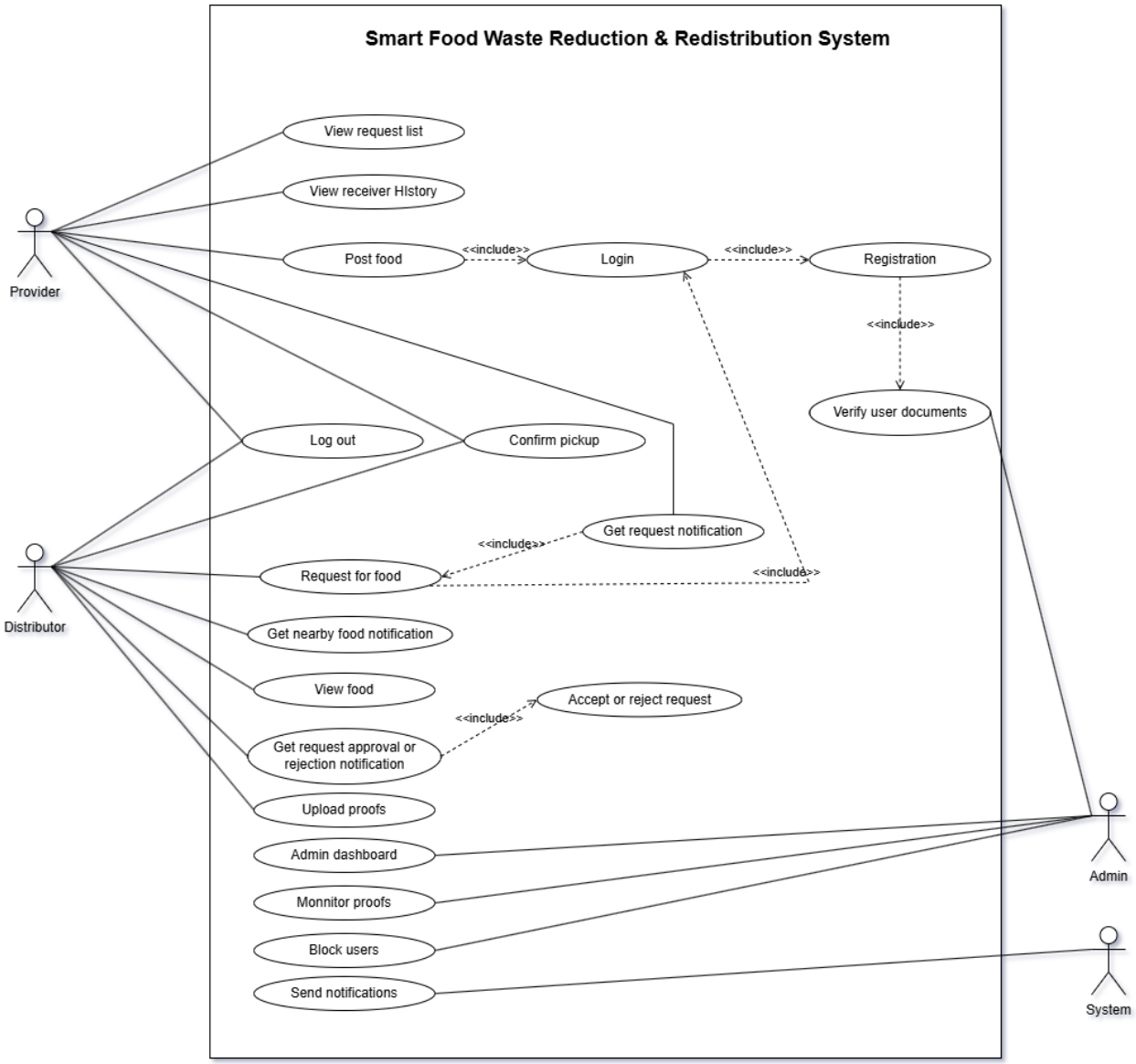
The system architecture shows how FoodShare's components work together. It includes a React frontend, a Node.js/Express backend, MongoDB, JWT authentication, and a real-time notification service.

This layered design supports scalability, security, and maintainability.

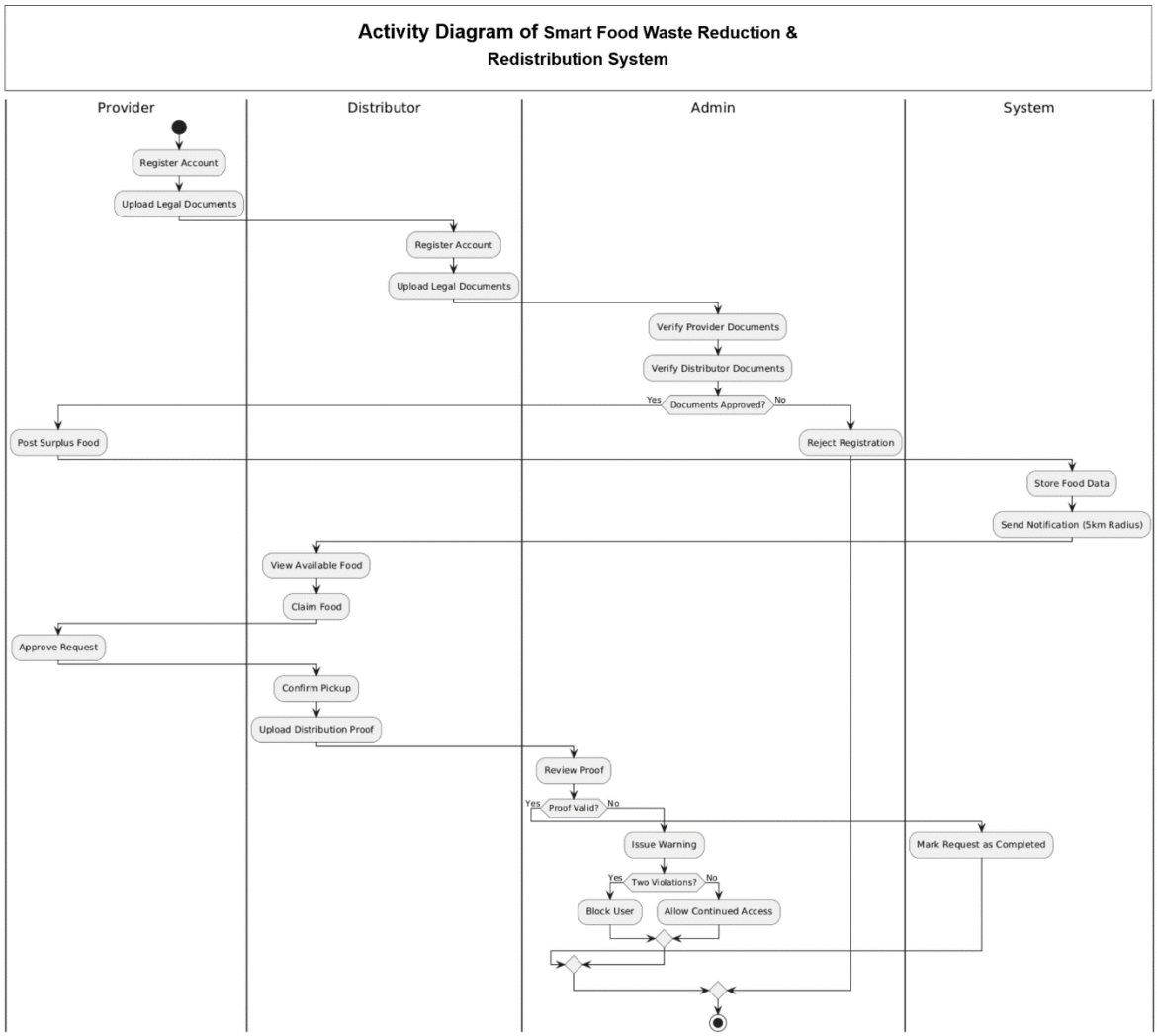


4.2. System Modeling

Use Case Diagram:

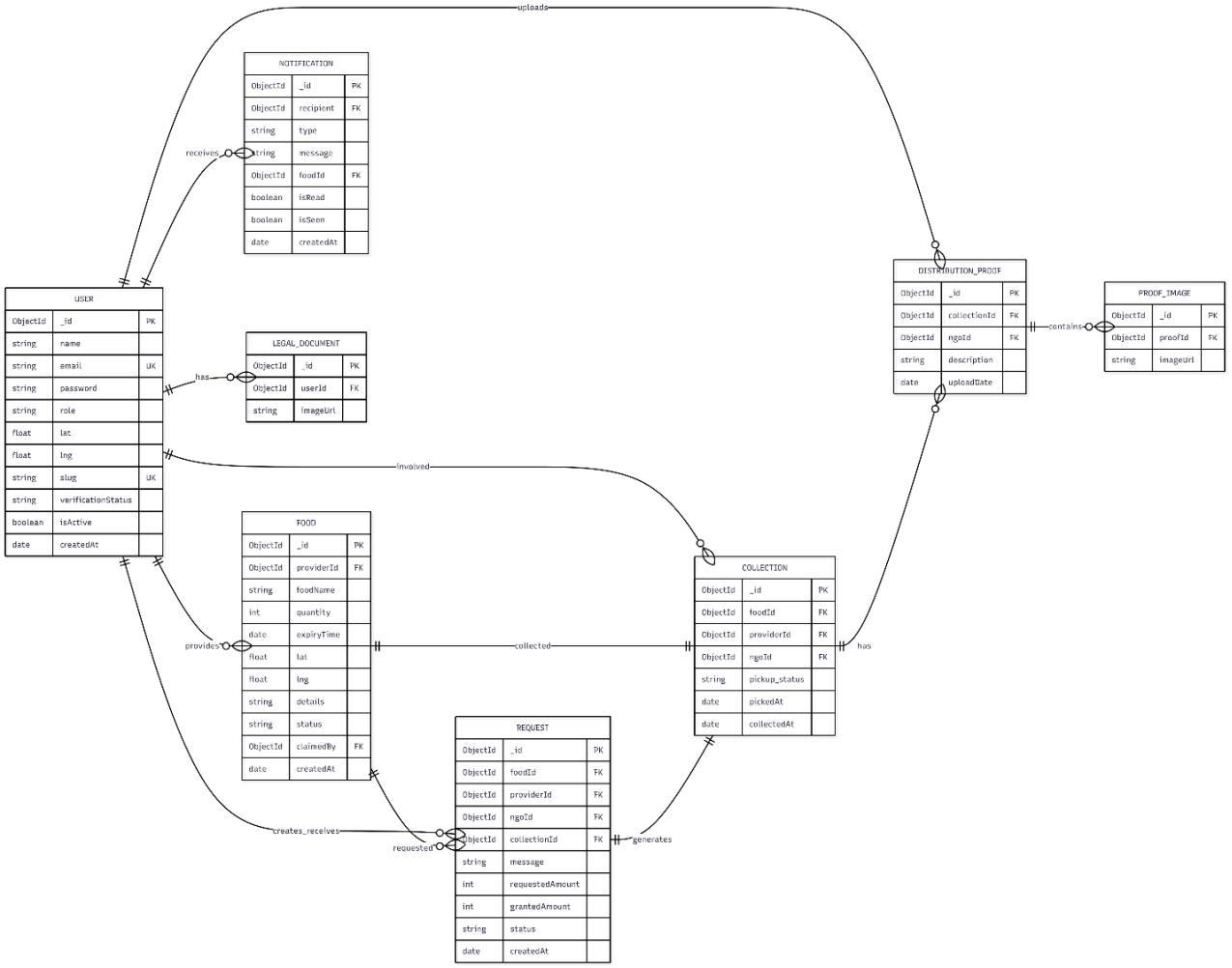


Activity Diagram:



4.3. Database Design

4.3.1. Entity Relationship Diagram (ERD):



4.3.2. Data Dictionary:

USER

Field	Type	Constraints
id	VARCHAR(24)	Primary Key
name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
password	VARCHAR(255)	NOT NULL
role	ENUM	provider/ngo/admin
lat	FLOAT	DEFAULT 0
lng	FLOAT	DEFAULT 0
slug	VARCHAR(150)	UNIQUE
verificationStatus	ENUM	pending/approved/rejected
isActive	BOOLEAN	DEFAULT FALSE
createdAt	DATETIME	DEFAULT CURRENT_TIMESTAMP

FOOD

Field	Type	Constraints
id	VARCHAR(24)	Primary Key
providerId	VARCHAR(24)	FK
foodName	VARCHAR(100)	NOT NULL
quantity	INT	NOT NULL
expiryTime	DATETIME	NOT NULL
lat	FLOAT	
lng	FLOAT	
details	TEXT	
status	ENUM	available/claimed/...
claimedBy	VARCHAR(24)	FK
createdAt	DATETIME	DEFAULT CURRENT_TIMESTAMP

REQUEST

Field	Type	Constraints
id	VARCHAR(24)	Primary Key
foodId	VARCHAR(24)	FK
providerId	VARCHAR(24)	FK
ngoId	VARCHAR(24)	FK
collectionId	VARCHAR(24)	FK
message	TEXT	
requestedAmount	INT	DEFAULT 0
grantedAmount	INT	
status	ENUM	pending/accepted/...
createdAt	DATETIME	DEFAULT CURRENT_TIMESTAMP

COLLECTION

Field	Type	Constraints
id	VARCHAR(24)	Primary Key
foodId	VARCHAR(24)	FK
providerId	VARCHAR(24)	FK
ngoId	VARCHAR(24)	FK
pickup_status	ENUM	pending/completed
pickedAt	DATETIME	
collectedAt	DATETIME	DEFAULT CURRENT_TIMESTAMP

NOTIFICATION

Field	Type	Constraints
id	VARCHAR(24)	Primary Key
recipient	VARCHAR(24)	FK
type	ENUM	new_food/request_update
message	TEXT	NOT NULL

foodId	VARCHAR(24)	FK
isRead	BOOLEAN	DEFAULT FALSE
isSeen	BOOLEAN	DEFAULT FALSE
createdAt	DATETIME	DEFAULT CURRENT_TIMESTAMP

DISTRIBUTION_PROOF

Field	Type	Constraints
id	VARCHAR(24)	Primary Key
collectionId	VARCHAR(24)	FK
ngoId	VARCHAR(24)	FK
description	TEXT	
uploadDate	DATETIME	DEFAULT CURRENT_TIMESTAMP

LEGAL_DOCUMENT

Field	Type	Constraints
id	VARCHAR(24)	Primary Key
userId	VARCHAR(24)	FK
imageUrl	VARCHAR(255)	NOT NULL

PROOF_IMAGE

Field	Type	Constraints
id	VARCHAR(24)	Primary Key
proofId	VARCHAR(24)	FK
imageUrl	VARCHAR(255)	NOT NULL

5. Technical Implementation

5.1. Backend / API Documentation

5.1.1. Overview

The frontend communicates with the backend through a centralized API system. All API requests are handled using an Axios instance defined in api.js.

- The api.js file sets the base URL for backend communication.
- All frontend components use this centralized API instance for sending requests.
- Authentication is managed using JWT (JSON Web Token).
- Backend routes are organized into feature-based modules for better scalability.

5.1.2. API Communication Architecture

5.1.2.1. API Instance (api.js)

The application uses a single API configuration file.

Base URL:

- http://localhost:5000/ OR environment variable VITE_API_BASE_URL

This API instance is used throughout the frontend for:

- GET requests
- POST requests
- PUT requests
- DELETE requests

5.1.2.2. Authentication Handling

After successful login, the system generates a JWT token which is stored in the frontend.

This token is attached to all protected API requests.

Authorization Header Format:

- Authorization: Bearer <token>
- The token is retrieved using a helper function and included in request headers.
- This mechanism ensures secure access to protected routes.

Common Files Using Authentication:

- Login.jsx
- Register.jsx
- Profile-related components
- Dashboard components

5.1.3. API Endpoint Documentation

Authentication/User Management

Method	Endpoint	Purpose	Access Level	Frontend Component
POST	/api/auth/register	Register Provider or NGO with required documents	Public	Register.jsx
POST	/api/auth/login	User login and JWT generation	Public	Login.jsx
GET	/api/auth/me	Retrieve logged-in user profile	Authenticated	Profile.jsx
PUT	/api/auth/profile	Update user profile	Authenticated	Profile.jsx
GET	/api/auth/profile/:id	View another user profile	Authenticated	Profile.jsx, Home.jsx, Foods.jsx

Food Post Management

Method	Endpoint	Purpose	Access Level	Frontend Component
POST	/api/food/create	Create a surplus food post	Provider	ProviderDashboard.jsx
GET	/api/food/my-food	Get provider's own food posts	Provider	ProviderDashboard.jsx
GET	/api/food/available	Get available food listings	Public	Home.jsx, Foods.jsx
DELETE	/api/food/:id	Delete a food post	Provider	ProviderDashboard.jsx

Provider Request Management

Method	Endpoint	Purpose	Access Level	Frontend Component
GET	/api/provider/requests	Get incoming requests	Provider	ProviderDashboard.jsx
GET	/api/provider/distribution-proofs	View distribution proofs	Provider	ProviderDashboard.jsx
PUT	/api/provider/request/:id/accept	Accept request	Provider	ProviderDashboard.jsx
PUT	/api/provider/request/:id/reject	Reject request	Provider	ProviderDashboard.jsx

NGO/Distributor Operations

Method	Endpoint	Purpose	Access Level	Frontend Component
GET	/api/ngo/nearby	Get nearby available food	NGO	NGODashboard.jsx
PUT	/api/ngo/claim/:id	Claim food post	NGO	Backend only (not used in frontend)
PUT	/api/ngo/collect/:id	Mark food as collected	NGO	Backend only (not used in frontend)
PUT	/api/ngo/collection/:id/complete	Complete pickup process	NGO	NGODashboard.jsx
POST	/api/ngo/request/:foodId	Send pickup request	NGO	NGODashboard.jsx, Home.jsx, Foods.jsx
GET	/api/ngo/requests	Get NGO requests	NGO	NGODashboard.jsx
GET	/api/ngo/collections	Get NGO collections	NGO	Not currently used in frontend
POST	/api/ngo/distribution-proof/:collectionId	Upload distribution proof	NGO	NGODashboard.jsx
GET	/api/ngo/distribution-proofs	View uploaded proofs	NGO	NGODashboard.jsx

Admin Panel APIS

Method	Endpoint	Purpose	Access Level	Frontend Component
GET	/api/admin/users	Retrieve all users	Admin	AdminDashboard.jsx
GET	/api/admin/pending-users	View pending user verification	Admin	AdminDashboard.jsx
PUT	/api/admin/user/:id/verify	Approve or reject user	Admin	AdminDashboard.jsx
PUT	/api/admin/user/:id/block	Block or unblock user	Admin	AdminDashboard.jsx
DELETE	/api/admin/user/:id/delete	Delete user account	Admin	AdminDashboard.jsx
GET	/api/admin/foods	Retrieve all food posts	Admin	AdminDashboard.jsx
DELETE	/api/admin/food/:id	Delete food post	Admin	AdminDashboard.jsx
GET	/api/admin/stats	Get system statistics	Admin	AdminDashboard.jsx

GET	/api/admin/distribution-proofs	Get all distribution proofs	Admin	AdminDashboard.jsx
GET	/api/admin/user/:id/profile	View user profile details	Admin	Backend only
GET	/api/admin/red-alert-ngos	Get NGOs with overdue proofs	Admin	AdminDashboard.jsx

Notification System

Method	Endpoint	Purpose	Access Level	Frontend Component
GET	/api/notifications	Fetch notifications	Authenticated	NotificationBell.jsx
PUT	/api/notifications/seen	Mark all notifications as seen	Authenticated	NotificationBell.jsx
PUT	/api/notifications/:id/read	Mark single notification as read	Authenticated	NotificationBell.jsx

5.1.4. Data Flow Description

The system follows a structured API communication flow:

1. The api.js file acts as the central communication layer.
2. User actions in the frontend trigger API requests.
3. Backend processes requests and returns JSON responses.
4. The frontend updates UI based on the received data.

Common Response Data Models:

- User
- Food
- Request
- Collection
- Distribution Proof
- Notification

5.1.5. File Upload Handling

File uploads are managed through dedicated frontend components.

- ImageUploader.jsx
- DocumentUploader.jsx

Uploaded files are sent to the ImgBB service.

The service returns a public URL which is then stored in the backend database as part of the request payload.

5.1.6. Summary

This API architecture ensures:

- Centralized communication via api.js
- Secure authentication using JWT
- Modular backend route structure
- Clear separation of roles (Provider, NGO, Admin)
- Scalable and maintainable system design

5.2. Role-Based Access Control (RBAC)

The system enforces three distinct roles — Provider, Distributor, and Admin — at multiple levels of the application.

Backend Enforcement

Upon login, the server signs a JWT token that embeds the user's id, role, and isVerified status in its payload. Every protected API route passes through two middleware functions before the route handler executes:

1. verifyToken middleware: Extracts and validates the JWT from the Authorization header. If the token is missing, expired, or invalid, the request is rejected with a 401 Unauthorized response before it reaches the route.

2. authorizeRole(...roles) middleware: Checks the role field decoded from the token against the list of permitted roles for that specific route. If the user's role is not in the permitted list, the request is rejected with a 403 Forbidden response.

Additionally, a checkBlocked middleware verifies that the user's isBlocked flag is false before allowing any action. Blocked Distributors receive a 403 response on all protected routes.

Frontend Enforcement

On the client side, after a successful login the JWT is decoded and the user's role is stored in application state. React Router uses this role to direct users to their respective route:

- /provider → Provider Dashboard (food posting, request management)
- /ngo → Distributor Dashboard (nearby food, requests, proofs)
- /admin → Admin Dashboard (user management, statistics, proof monitoring)

Any attempt to manually navigate to a route outside of one's role redirects the user back to their own dashboard, preventing unauthorized UI access.

5.3. Frontend Organization

Frontend Structure Overview

The frontend of the Food Waste & Redistribution System is built using React (with Vite) and styled using Tailwind CSS. The project follows a modular, component-based architecture to ensure scalability, maintainability, and clean separation of concerns.

The main structure is organized as follows:

- **src/components/**
  Contains reusable UI components grouped by functionality:
  - Common/: Shared UI elements such as Footer, NotificationBell, ScrollToTop
  - Uploaders/: File upload components like ImageUploader and DocumentUploader
- **src/pages/**
  Contains page-level components mapped to application routes:
  - Example: Login.jsx, AdminDashboard.jsx
- **src/context/**
  Manages global state using React Context API
  - Example: ModalContext.jsx for centralized modal handling
- **src/utils/**
  Includes helper functions such as authentication utilities
- **Root Files**
  - App.jsx: Main application component
  - main.jsx: Entry point of the React app

This structure ensures clear separation between UI components, business logic, and routing, making the frontend easy to scale and maintain.

Component Reusability (Atomic Design Approach)

Component reusability is achieved using a structure inspired by Atomic Design principles, where UI components are divided into hierarchical levels:

Atoms (Basic Elements)

Small, reusable UI elements such as buttons, icons, and simple widgets.

Example: Components in the Common/ folder like Footer or NotificationBell.

Molecules (Combination of Atoms)

Groups of atoms forming functional units.

Example: Components in Uploaders/ such as ImageUploader, which combine input fields and buttons.

Organisms (Complex Components)

Larger UI sections composed of multiple atoms and molecules.

Example: DashboardLayout.jsx, which provides a reusable layout (sidebar, header) for all dashboard pages.

Pages

Complete UI views composed of organisms and smaller components.

Example: AdminDashboard.jsx, Login.jsx.

How Reusability is Achieved

- **Modular Component Design**

Components are created once and reused across multiple pages using imports.

- **Shared Layouts**

Layout components (e.g., DashboardLayout) eliminate repetition across pages.

- **Global State Management**

Context API (ModalContext) allows shared functionality across components.

- **Utility Functions**

Common logic (e.g., authentication) is centralized in utils to avoid duplication.

- **Consistent Styling with Tailwind CSS**

Reusable utility classes ensure consistent design without redundant CSS.

6. User Manual

6.1. Getting Started

Registration

1. Navigate to the platform's home page and click Register.
2. Fill in your full name, email address, and password.
3. Select your role — Provider or Distributor (NGO).
4. Upload your required legal documents for admin verification.
5. Submit the form. Your account will be created with a Pending Verification status.
6. Wait for the Admin to review and verify your documents. You will gain full access once verified.

Note: Admin accounts are not self-registered. Admin credentials are pre-configured at the system level.

Login

1. Navigate to the Login page.
2. Enter your registered email address and password.
3. Click Login.
4. The system validates your credentials and role. Upon success, you are automatically redirected to your role-specific dashboard:
   - Providers → /provider — Provider Dashboard
   - Distributors → /ngo — NGO/Distributor Dashboard
   - Admin → /admin — Admin Dashboard
5. If your account is pending verification or has been blocked, an appropriate error message is displayed and access is denied.

Navigating the Dashboard

Each dashboard presents only the features relevant to that role. The top navigation bar provides quick access to all major sections. To log out at any time, click the Logout

button in the navigation bar. Your session is immediately terminated and you are redirected to the Login page.

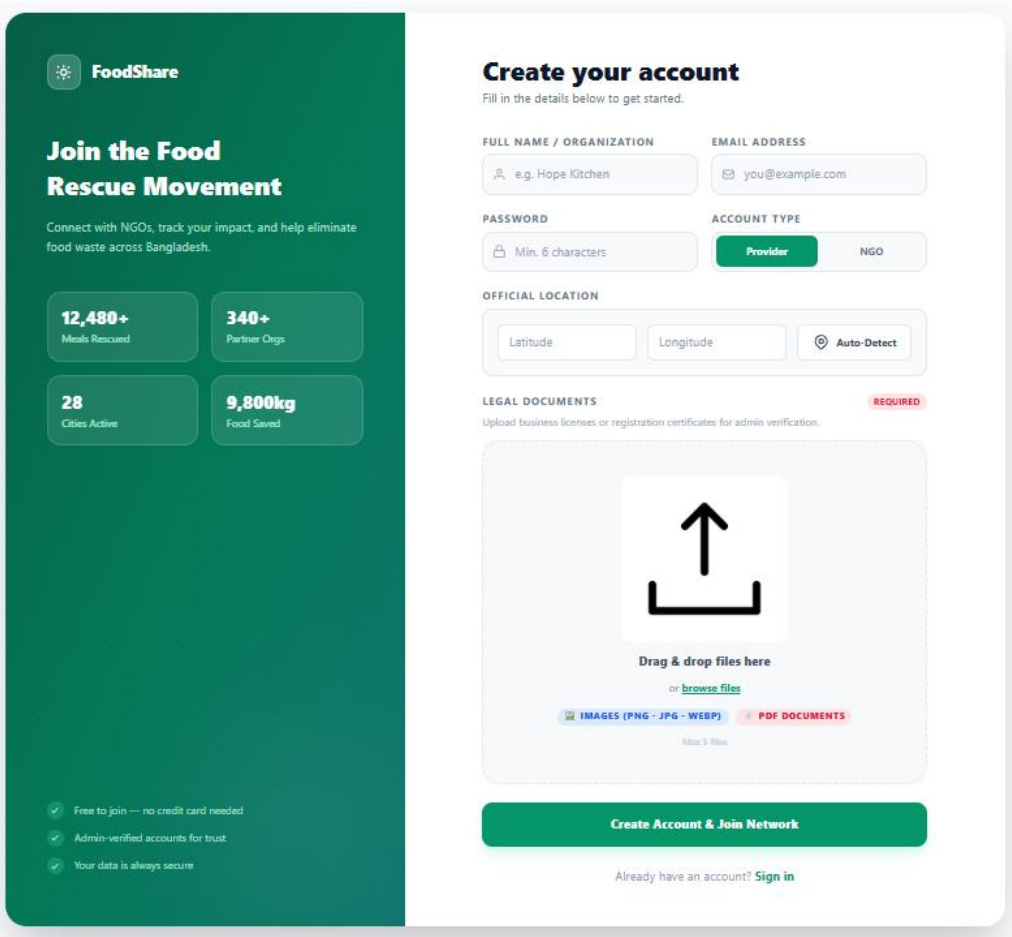
6.2. Feature Walkthroughs

FR-1 & FR-2 — Registration & Document Submission

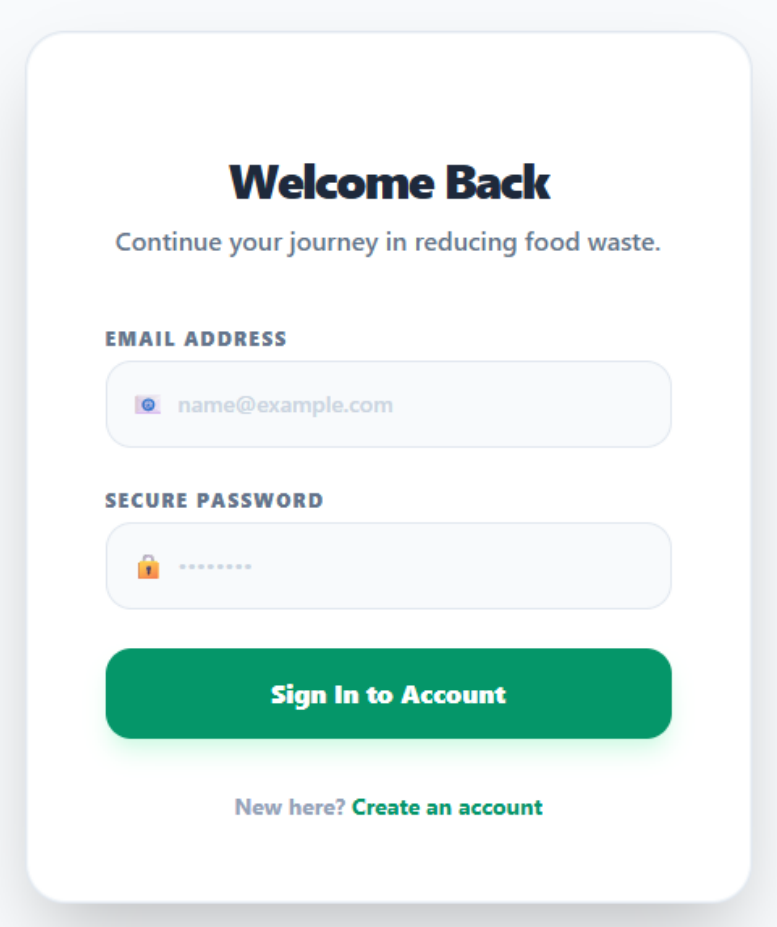
Who: Provider, Distributor

Steps:

1. Go to the Register page.
2. Enter your name, email, password, and select your role.
3. In the document upload section, attach your legal document (e.g. trade license, NGO registration certificate).
4. Click Submit Registration.
5. The system creates your account and stores your document with a Pending status.
6. You will see a confirmation message informing you that your account is under review.



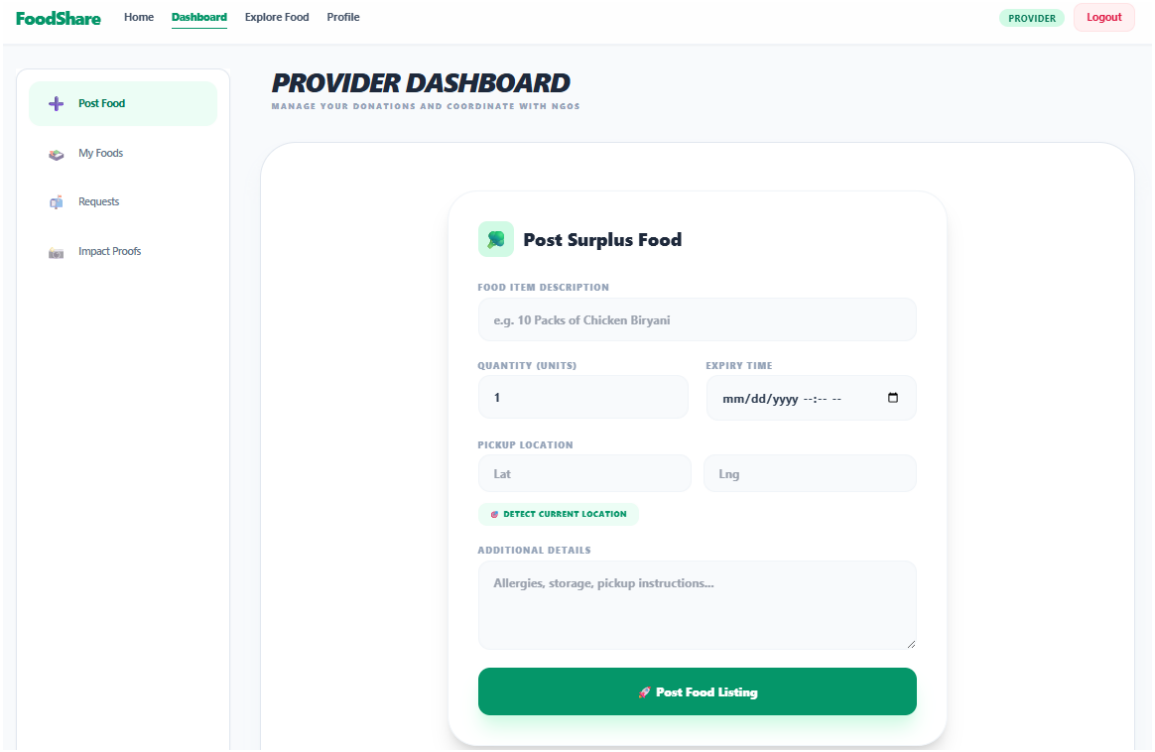
FR-3 — Login
Who: Provider, Distributor, Admin
Steps:
1. Go to the Login page.
2. Enter your email and password.
3. Click Login.
4. On success, you are redirected to your role-based dashboard automatically.
5. On failure (wrong credentials, blocked account, unverified account), an error message is displayed.



FR-4 — Role-Based Dashboard
Who: Provider, Distributor, Admin

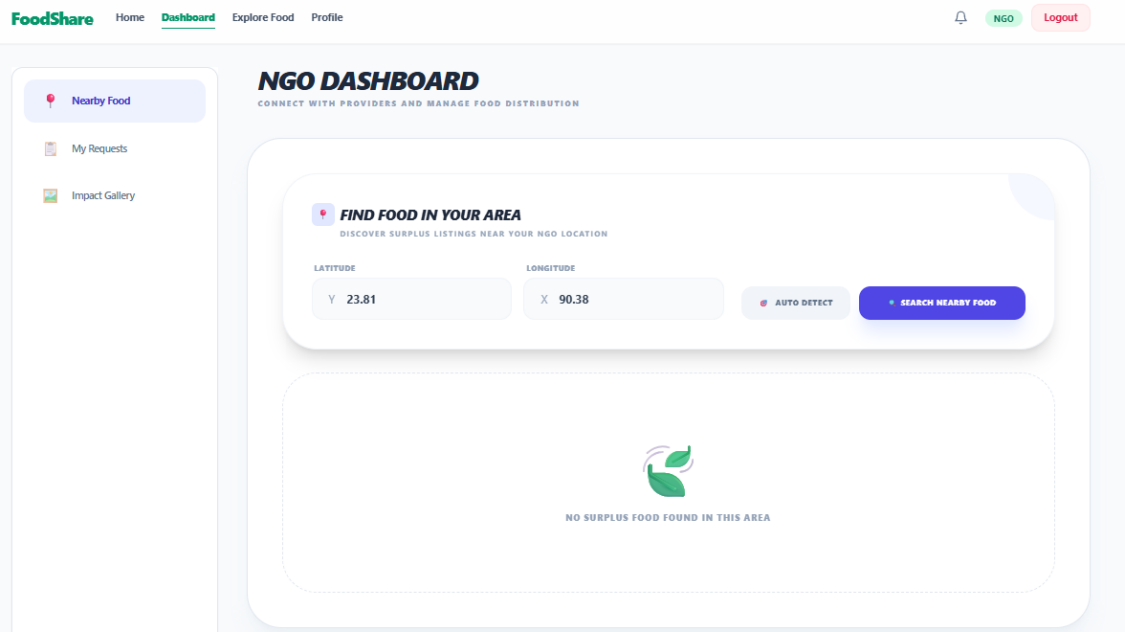
Provider Dashboard:

1. After login, you land on the Provider Dashboard.
2. The dashboard displays your active food posts, incoming pickup requests, and transaction history.
3. Use the navigation menu to switch between posting food, managing requests, and viewing history.



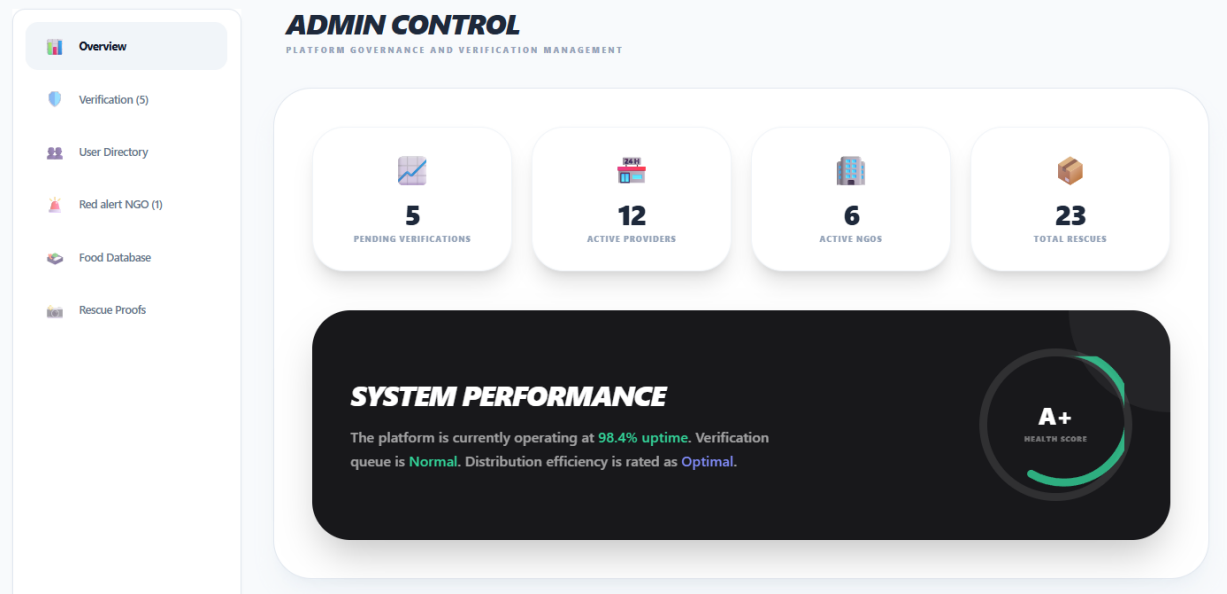
Distributor (NGO) Dashboard:

1. After login, you land on the NGO Dashboard.
2. The dashboard displays nearby available food, your active requests, and your proof submission history.
3. Use the navigation menu to browse food, manage your requests, and upload proofs.



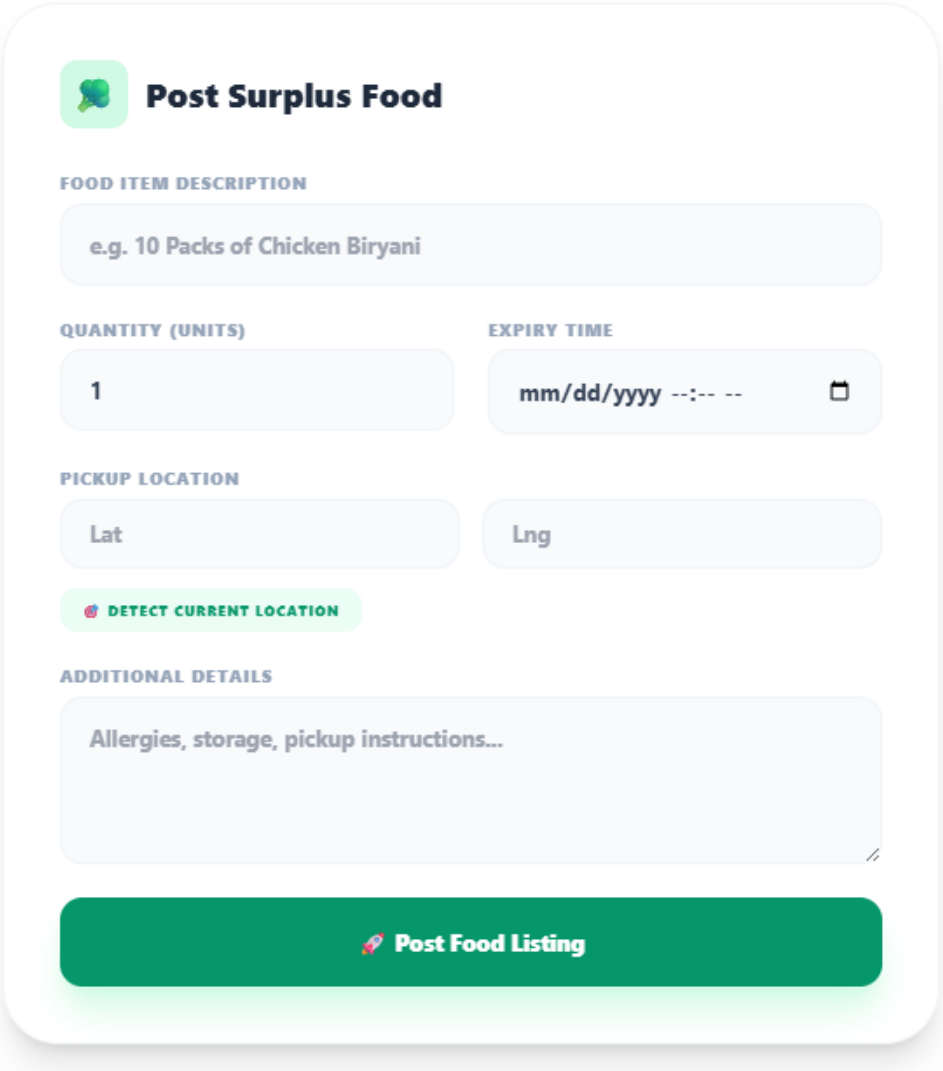
Admin Dashboard:

1. After login, you land on the Admin Dashboard.
2. The dashboard shows platform-wide statistics, pending document verifications, food listings, and distribution proofs awaiting review.



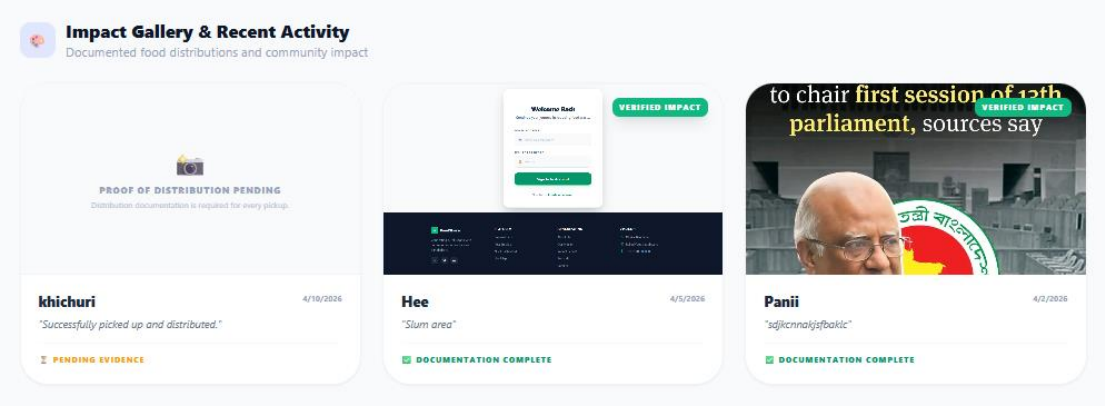
FR-5 — Post Food (Provider)
Who: Provider
Steps:

1. From the Provider Dashboard, click Post Food.
2. Fill in the food details form:
   - Food Type — name or description of the food
   - Quantity — amount or number of servings
   - Expiry Time — the deadline by which the food must be collected
   - Location — your pickup address or pin on the map
3. Click Submit Post.
4. The food post is saved and becomes visible to nearby Distributors.
5. The system attempts to notify Distributors within 5km of the posting location.



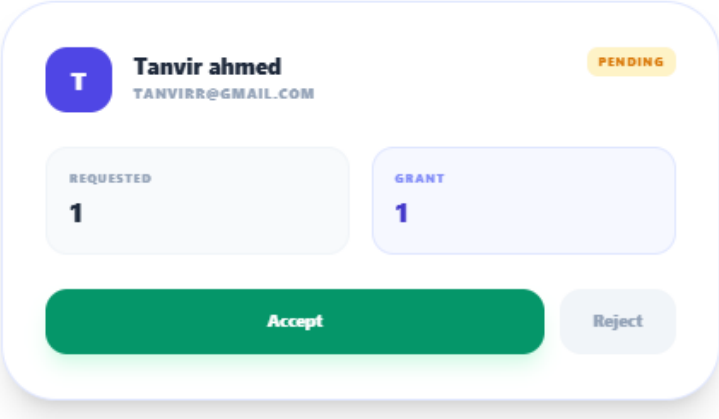
FR-7 — View Distributor History (Provider)
Who: Provider
Steps:

1. From the Provider Dashboard, navigate to Incoming Requests.
2. Click on any pending request to expand it.
3. Click View Distributor History to see the selected Distributor's past transactions and distribution proofs.
4. Review the history to assess the Distributor's reliability before making a decision.



FR-8 — Accept or Reject Request (Provider)
Who: Provider
Steps:

1. From the Provider Dashboard, navigate to Incoming Requests.
2. Each request displays the Distributor's name and submission time.
3. Click View History to review the Distributor's track record if needed.
4. Click Approve to accept the request or Reject to decline it.
5. The system updates the request status and notifies the Distributor of the decision.

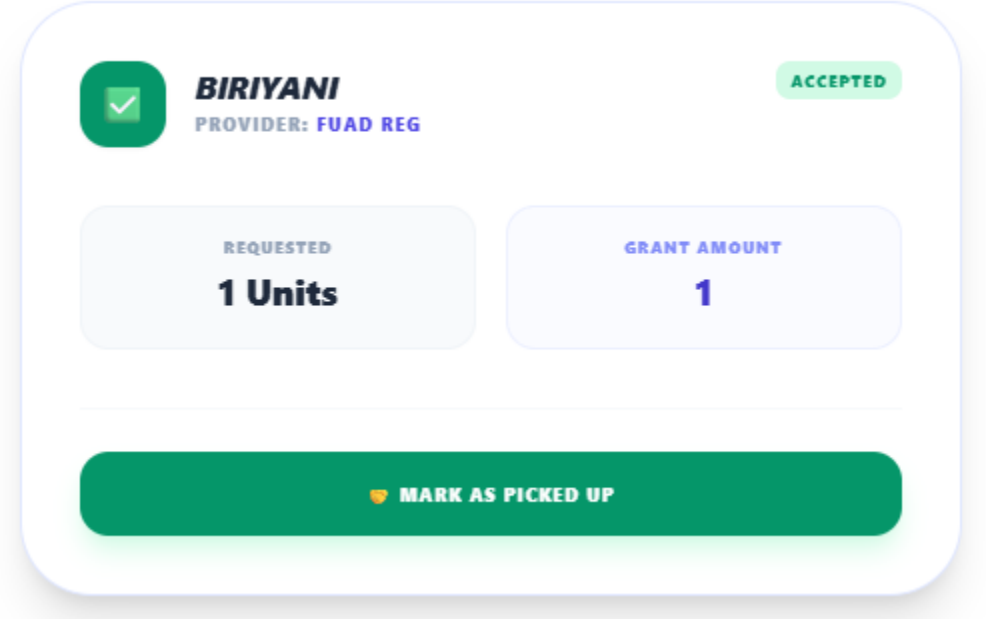


FR-9 — Confirm Pickup (Provider & Distributor)
Who: Provider, Distributor
Steps (Distributor side):

1. Once your request has been approved, go to My Requests in the NGO Dashboard.
2. Locate the approved request and click Confirm Pickup after physically collecting the food.

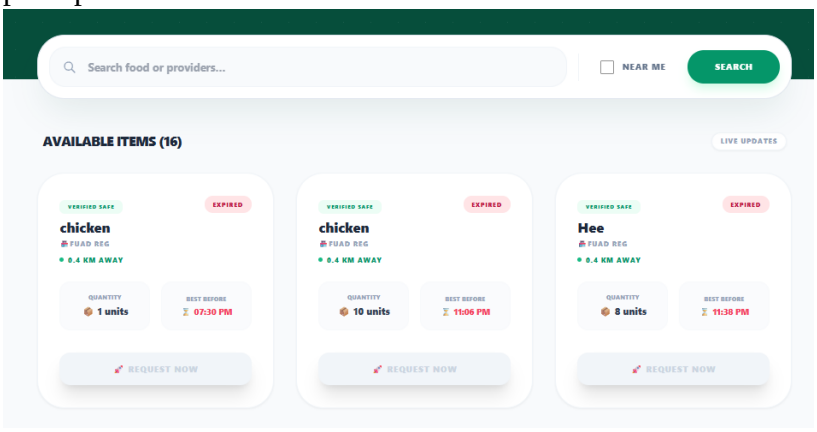
Steps (Provider side):

1. In the Provider Dashboard, go to Active Requests.
2. Once the Distributor has confirmed, click Confirm Pickup on your side to finalize the transaction.
3. The system marks the transaction as Completed and logs the record.



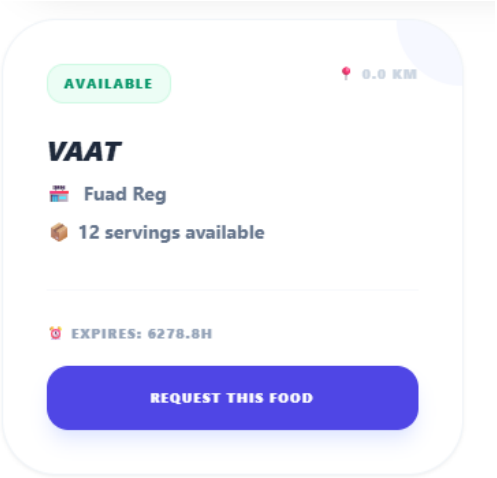
FR-12 — View Food & Search (Distributor)
Who: Distributor
Steps:

1. From the NGO Dashboard, navigate to the Food Listings page (/foods).
2. Browse the list of all currently available food posts.
3. Use the Search Bar to search by food type or provider name.
4. Use the Filters to narrow results by location or expiry time.
5. Click on any food post to view its full details including quantity, expiry, and pickup location.



FR-13 — Request for Food (Distributor)
Who: Distributor
Steps:

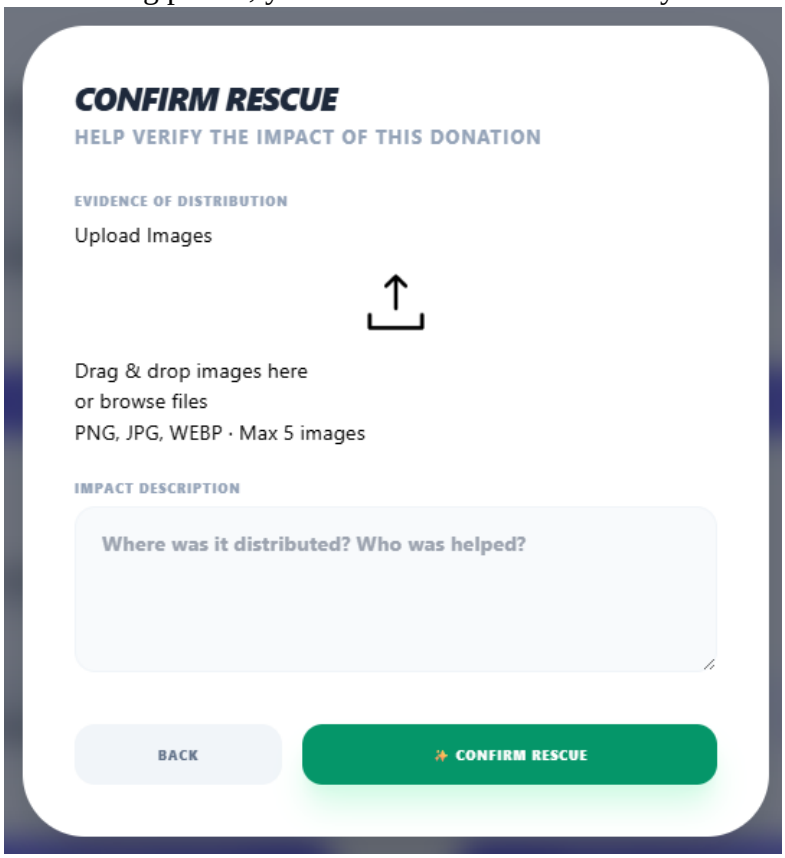
1. From the Food Listings page, select the food post you wish to collect.
2. Click Request Pickup.
3. The system records your request and sets its status to Pending.
4. The Provider is notified of your request and will review your history before responding.
5. You can monitor the status of your request from My Requests in the NGO Dashboard.



FR-15 — Upload Distribution Proof (Distributor)
Who: Distributor
Steps:

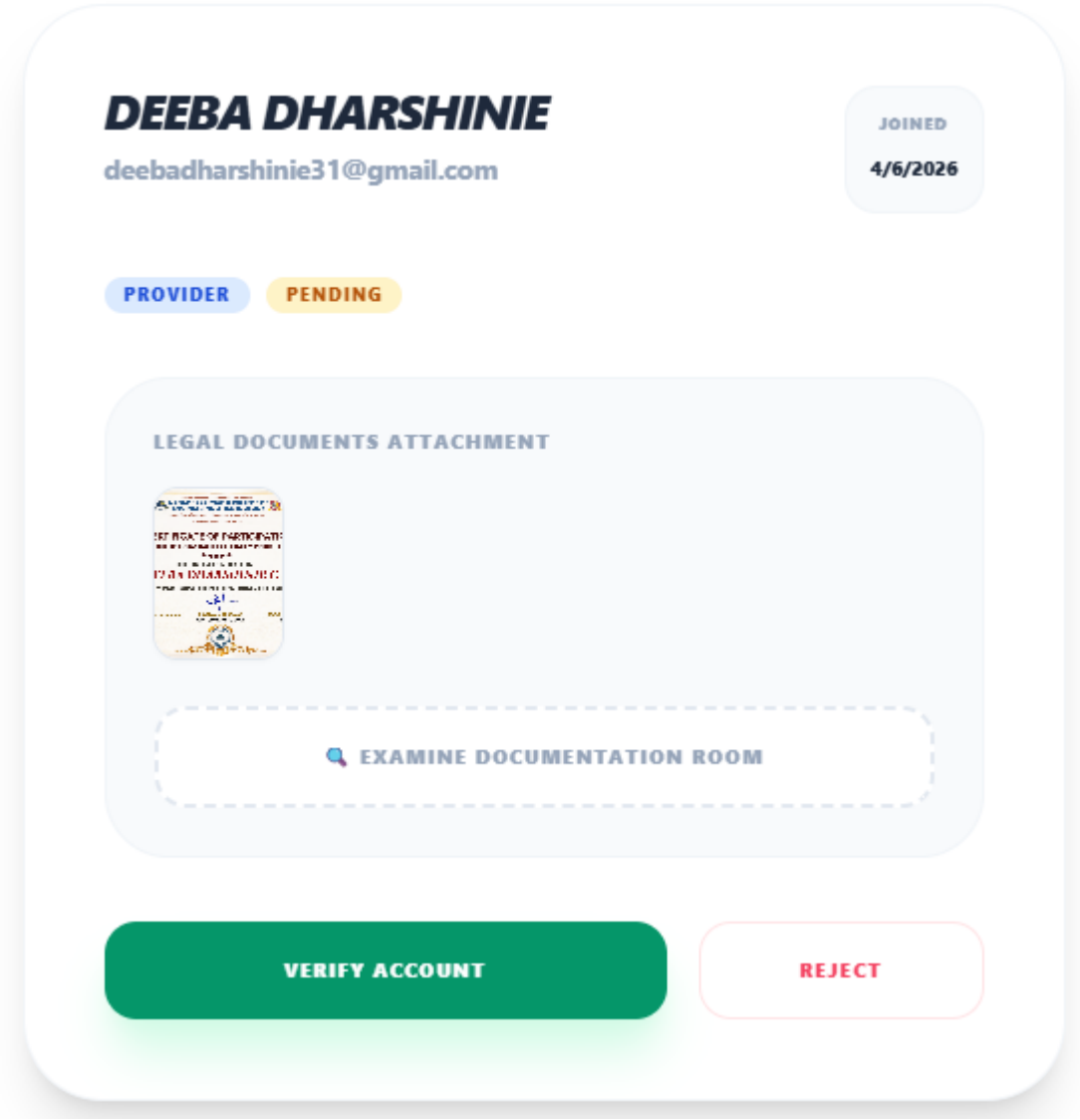
1. After completing a food delivery, go to My Requests in the NGO Dashboard.
2. Locate the completed transaction and click Upload Proof.
3. Attach a photo or written report as evidence of distribution.
4. Click Submit Proof.
5. The proof is stored with a Pending Review status and forwarded to the Admin for verification.

Important: Failing to upload a proof for a completed transaction is recorded. After two missing proofs, your account will be blocked by the Admin.



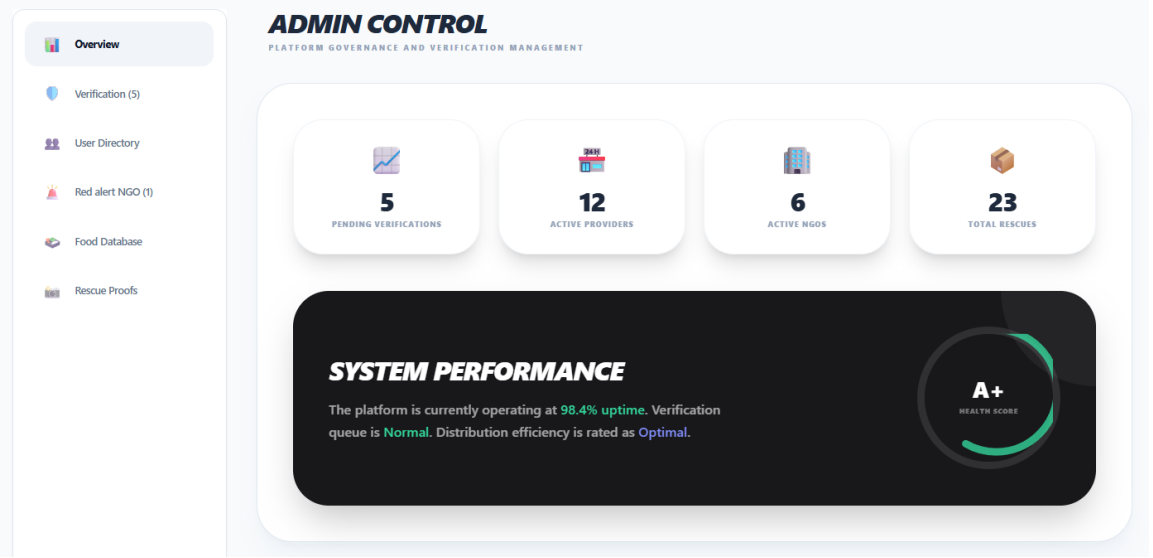
FR-16 — Verify User Documents (Admin)
Who: Admin
Steps:

1. From the Admin Dashboard, navigate to Pending Verifications.
2. Click on a user entry to view their submitted legal documents.
3. Review the documents carefully.
4. Click Verify to approve the user and grant them full platform access, or Reject to deny their registration.
5. The user's account status is updated immediately.



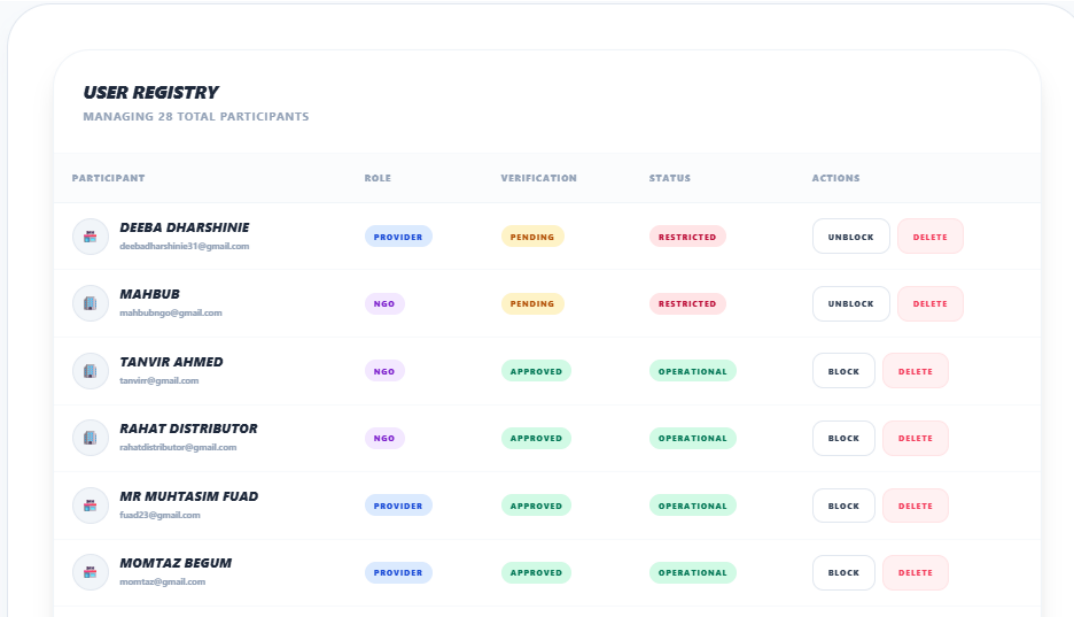
FR-17 — Admin Dashboard
Who: Admin
Steps:

1. After login, you are taken directly to the Admin Dashboard.
2. The dashboard displays:
   - Statistics Panel — total users, total food posts, completed transactions, pending verifications
   - Food Listings — all active and past food posts across the platform
   - User Management — list of all registered users with their verification and block status
3. Use the navigation tabs to switch between sections.



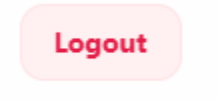
FR-18 & FR-19 — Monitor Proofs & Block Users (Admin)
Who: Admin
Steps:

1. From the Admin Dashboard, navigate to Distribution Proofs.
2. Browse the list of submitted proofs with their current status (Pending, Verified, Rejected).
3. Click on a proof entry to view the uploaded photo or report.
4. Click Verify if the proof is valid or Reject if it is fraudulent or insufficient.
5. The system tracks each Distributor's missing proof count automatically.
6. When a Distributor accumulates two rejected or missing proofs, navigate to User Management, locate the user, and click Block User.
7. The blocked Distributor is immediately denied access to all platform features.



FR-10 — Logout
Who: Provider, Distributor, Admin
Steps:

1. Click the Logout button in the top navigation bar from any page.
2. Your JWT session token is invalidated.
3. You are redirected to the Login page immediately.



7. Quality Assurance & Testing

7.1. Test Case Scenarios

Test ID	Test Description	Input Data	Expected Result	Actual Result	Status
TC-01	User Registration	Valid provider/distributor data	Account created successfully	Account created	Pass
TC-02	Document Submission	Upload valid legal documents	Documents stored & sent for admin verification	Successfully submitted	Pass
TC-03	User Login	Correct email & password	User redirected to role-based dashboard	Login successful	Pass
TC-04	Role-Based Dashboard	Login as Provider/Distributor/Admin	Role base dashboard displayed	Correct dashboard shown	Pass
TC-05	Post Food	Food type, quantity, location, expiry	Food post created and visible	Successfully posted	Pass
TC-06	View Food Listings	Distributor browsing	Show food list	Listings displayed	Pass
TC-07	Request Food	Distributor submits request	Request sent to provider	Request created	Pass
TC-08	View Distributor History	Provider checks distributor	Transaction history displayed	Data shown correctly	Pass
TC-09	Accept/Reject Request	Provider decision input	Request status updated	Status updated	Pass
TC-10	Confirm Pickup	Both confirm pickup	Status marked as completed	Completed successfully	Pass
TC-11	Upload Proof	Distributor uploads image/report	Proof stored in system	Uploaded successfully	Pass
TC-12	Admin Verify Documents	Admin reviews docs	User verified or rejected	Verified correctly	Pass
TC-13	Block User	Distributor missing proofs ≥ 2	User blocked	User blocked	Pass
TC-14	Logout	Click logout	Session terminated	Logout successful	Pass
TC-15	Nearby Food Notification	Food within 5km	Notification sent	Partial notification	Pass

TC-16	Request Status Notification	Provider approves/rejects	Distributor notified	Partial notification	Pass
TC-17	Provider Request Notification	Distributor sends request	Provider gets notification	Not received	Pass
TC-18	Automated Notifications	System triggers alerts	Notifications sent	Not implemented	Pass

7.2. Requirements Traceability Matrix (RTM)

Req ID	Requirement	Test Case ID(s)	Coverage Status
FR-1	Registration	TC-01	Covered
FR-2	Submit Documents	TC-02	Covered
FR-3	Login	TC-03	Covered
FR-4	Role-Based Dashboard	TC-04	Covered
FR-5	Post Food	TC-05	Covered
FR-6	Provider Notification	TC-17	Covered
FR-7	View Distributor History	TC-08	Covered
FR-8	Accept/Reject Request	TC-09	Covered
FR-9	Confirm Pickup	TC-10	Covered
FR-10	Logout	TC-14	Covered
FR-11	Nearby Food Notification	TC-15	Covered
FR-12	View Food	TC-06	Covered
FR-13	Request Food	TC-07	Covered
FR-14	Request Status Notification	TC-16	Covered
FR-15	Upload Proof	TC-11	Covered
FR-16	Verify Documents	TC-12	Covered
FR-17	Admin Dashboard	TC-04	Covered
FR-18	Monitor Proofs	TC-11	Covered
FR-19	Block Users	TC-13	Covered
FR-20	Send Notifications	TC-18	Covered

8. Individual Contribution

Muhtasim Fuad Rahat (Backend Developer)

Technical Contribution

Module	File/Folder	Description
Database Models	models/User.js	Stores user data and roles (Provider, NGO, Admin)
	models/Food.js	Handles food listings with geolocation
	models/Request.js	Manages food request lifecycle
	models/DistributionProof.js	Stores proof of distribution
	models/Notification.js	Handles notifications system
Authentication APIs	routes/auth.js	Registration, login, logout functionality
Provider APIs	routes/provider.js	Food posting and request management
NGO APIs	routes/ngo.js	Food request and proof submission
Admin APIs	routes/admin.js	User verification and blocking
Notification APIs	routes/notification.js	Notification handling
Middleware	middleware/auth.js	JWT authentication
	middleware/roles.js	Role-Based Access Control
Utility	utils/geo.js	Geolocation using Haversine formula
Profile Management	pages/Profile.jsx	User profile handling (Profile – Rahat)
Frontend Support	pages/Login.jsx	Login UI
	pages/Register.jsx	Registration UI
	pages/AdminDashboard.jsx	Admin dashboard

Technical Challenges

Designing scalable schemas with geolocation (Food.js, User.js) was difficult, especially for nearby food queries. This was solved using MongoDB geospatial indexing and optimizing distance calculations. Handling concurrent requests required implementing Mongoose transactions and optimistic locking. Real-time notifications were challenging; API polling was used instead of WebSockets.

Learning Outcomes

Strengthened backend development skills with Express.js and MongoDB. Learned advanced database techniques such as geospatial queries and aggregation. Improved knowledge of secure authentication (JWT) and scalable architecture. Gained experience in API testing and backend structuring.

Md Mahbub Alam (Provider Frontend Developer & Tester)

Technical Contribution

Module	File/Folder	Description
Provider Dashboard	pages/ProviderDashboard.jsx	Role-based dashboard UI
Food Posting	components/Uploaders/ImageUploader.jsx	Handles food image uploads
Food Submission	pages/ProviderDashboard.jsx	Food submission form integrated with API
Request Management	pages/ProviderDashboard.jsx	View request history and distributor details; accept/reject requests
Pickup Confirmation	pages/ProviderDashboard.jsx	Pickup confirmation UI (buttons/modals)
Explore Food	pages/Foods.jsx / pages/Home.jsx	Browse and explore available food (Explore & Home – Mahbub)
Notifications UI	components/NotificationBell.jsx	Displays alerts and updates
Testing (UI & Performance)	Selenium & JMeter	Performed automated UI testing and load testing for system validation

Technical Challenges

Handling complex form validation caused UI lag, which was solved using React hooks and debouncing. Rendering large request datasets impacted performance, improved using pagination. API integration required proper error handling and loading states, implemented using try-catch and UI alerts.

During testing, ensuring stable Selenium test scripts across dynamic React components was challenging due to asynchronous rendering. This was resolved using proper waits and stable selectors. In JMeter testing, simulating realistic concurrent user load required careful configuration of thread groups and request timing to avoid unrealistic spikes.

Learning Outcomes

Learned React best practices including reusable components and efficient state management. Improved API integration and frontend performance optimization. Gained experience using Vite and Tailwind CSS for modern UI development.
Additionally, gained practical experience in software testing by using Selenium for automated UI testing and JMeter for performance and load testing. This helped understand real-world application stability, response time measurement, and scalability validation under concurrent user load.

Md Tanvir Ahmed (NGO/Distributor Frontend Developer& UI Designer)

Technical Contribution

Module	File/Folder	Description
NGO Dashboard	pages/NGODashboard.jsx	NGO-specific dashboard UI
Food Browsing	pages/Foods.jsx	Search and filter food listings
Request System	pages/NGODashboard.jsx	Request submission interface
Proof Upload	components/Uploaders/DocumentUploader.jsx	Upload document proofs
	components/Uploaders/ImageUploader.jsx	Upload image proofs
Notifications UI	components/NotificationBell.jsx	Displays request updates
Geolocation Alerts	pages/NGODashboard.jsx	Nearby food alerts
Navigation Bar	components/Navbar.jsx	Global navigation (Navbar – Tanvir)
Footer	components/Footer.jsx	Footer section across pages (Footer – Tanvir)

Technical Challenges

Search and filter features initially caused UI lag due to frequent API calls; solved using caching and debouncing. File upload handling required validation and preview control, implemented using React hooks. Geolocation permissions varied across devices, handled with fallback mechanisms.

Learning Outcomes

Enhanced skills in React hooks, API integration, and responsive design. Learned efficient frontend-backend communication and improved UX for forms and uploads. Gained practical experience in handling real-world UI challenges.

9. Conclusion & Future Work

9.1. Conclusion

The Smart Food Waste Reduction & Redistribution System successfully delivers a centralized, accountable web platform that replaces inefficient manual food redistribution processes with a structured digital workflow. Out of 20 defined functional requirements, 16 were fully implemented, 2 were partially implemented (geolocation-based nearby food detection and request status notifications), and 2 remain as future enhancements (full push notification module and provider-side request notifications). The core redistribution workflow — from food posting and pickup requesting to proof submission and admin monitoring — is fully functional and live at https://foodrescuebd.vercel.app.

The platform's JWT-based authentication, role-based access control, and mandatory proof submission system introduce a level of transparency and accountability entirely absent from existing manual approaches. Load testing confirmed stable performance under 100 concurrent users, validating the platform's readiness for real-world use. The project stands as a complete, deployable solution with meaningful potential impact on food waste reduction and community welfare.

9.2. Future Enhancements

1. Mobile Application: Develop a React Native mobile app to support Distributors who are on the move during food collection and delivery.

2. Live Delivery Tracking: Integrate real-time GPS tracking to give Providers end-to-end visibility from pickup to delivery completion.

3. Automated Proof Validation: Explore AI-assisted image recognition to automatically validate distribution proof photos, reducing manual Admin workload.

4. Analytics & Reporting Dashboard: Add visual charts and exportable reports on redistribution volumes, top Distributors, and high-activity zones to support NGO and policy-level decision making.

5. Multi-Language Support: Add Bangla language support to make the platform accessible to a wider range of local users across Bangladesh.

6. Automated Blocking System: Automate the Distributor blocking process so the system enforces violations without requiring manual Admin intervention, including automated warning emails after the first missed proof.

7. Point Reward System: The system awards points to providers and distributors for each successful food donation and distribution, encouraging active participation and promoting consistent engagement.

10. Appendices

Appendix A:

https://github.com/fuad-rahat/Food_Waste_And_Redistribution_System/

Appendix B:

Installation & Setup Guide (Command-line Instructions to Get the Project Running)

This section provides step-by-step instructions to set up and run the project locally using command-line tools. Follow each step carefully to ensure proper configuration of both backend and frontend environments.

1. Clone the Repository

First, download the project from GitHub and navigate into the project directory:

git clone https://github.com/fuad-rahat/Food_Waste_And_Redistribution_System.git

cd Food_Waste_And_Redistribution_System

This will create a local copy of the project on your machine.

2. Backend Setup

Move into the backend folder and install all required dependencies:

cd backend

npm install

Next, create a .env file in the backend directory and configure the following environment variables:

PORT=5000

MONGODB_URI=your_mongodb_connection_string

JWT_SECRET=your_jwt_secret_key

PORT: Defines the server port.

MONGODB_URI: Connection string for MongoDB (Atlas or local).

JWT_SECRET: Secret key used for authentication.

After configuration, start the backend server:

npm run dev

The backend API will now run locally and handle all server-side operations.

3. Frontend Setup

Open a new terminal, navigate to the frontend folder, and install dependencies:

Cd ../frontend

npm install

Create a .env file in the frontend directory and add:

VITE_API_BASE_URL=https://foodrescuebd-backend.vercel.app

VITE_IMGBB_API_KEY=your_imgbb_api_key

VITE_API_BASE_URL: URL of the backend API.

VITE_IMGBB_API_KEY: API key for image upload functionality.

Then start the frontend development server:

npm run dev

This will launch the React application with hot-reload support.

4. Access the Application

Once both servers are running:

Frontend: http://localhost:5173

Backend API: http://localhost:5000

You can now access the system through the browser and interact with all features including authentication, dashboards, and food management.